# OPTIMAL CONTROL OF THE FUTURE VIA PROSPECTIVE FORAGING

**Anonymous authors**
Paper under double-blind review

## ABSTRACT

Optimal control of the future is the next frontier for AI. Current approaches to this problem are typically rooted in either reinforcement learning or online learning. While powerful, these frameworks for learning are mathematically distinct from Probably Approximately Correct (PAC) learning, which has been the workhorse for the recent technological achievements in AI. We therefore build on prior work on prospective learning, an extension of PAC learning (without control) in non-stationary environments (De Silva et al., 2023; 2024; Bai et al., 2026). Here, we further extend the PAC learning framework to address learning and *control* in non-stationary environments. Using this framework, called "Prospective Control", we prove that under certain fairly general assumptions, empirical risk minimization (ERM) asymptotically achieves the Bayes optimal policy. We then take a specific instance of prospective control, foraging—which is a canonical task for any mobile agent—be it natural or artificial. We illustrate that existing reinforcement learning algorithms fail to learn in these non-stationary environments, and even with modifications they are orders of magnitude less efficient than our prospective foraging agents.

## 1 INTRODUCTION

When building real-world machine learning and/or artificial intelligence ML/AI systems, we almost always care about their future performance, because we will be using them in the future. And yet, many classical formulations of ML/AI problems essentially ignore this fact. We therefore put it in, from the start. We explicitly build on Probably Approximately Correct (PAC) learning (Mohri et al., 2012). We recently introduced Prospective Learning (PL) (De Silva et al., 2023; 2024; Bai et al., 2026), which generalizes PAC learning to address stochastic processes, rather than independent and identically distributed data. Here, we generalize further to incorporate stochastic decision processes.

Reinforcement learning (RL) and online learning (OL) are two popular machine learing paradigms that focus on sequential decision making. RL typically models the environment as a Markov Decision Process (MDP) with states, actions, rewards, and aims to learn a behavior/policy that maximizes long-term return (Bertsekas, 2023; Sutton and Barto, 2018; Strehl et al., 2009). RL has achieved many notable real-world successes, such as Go (Silver et al., 2016). However, standard RL methods fail in non-stationary reward settings, and also require multiple episodes (though see (Chen et al., 2022)). On the other hand, online learning algorithms are run in a sequence of consecutive rounds, making a prediction, observing feedback, and updating (Shalev-Shwartz, 2012). The goal of online learning algorithms is to find the best and fixed hypothesis in hindsight, and therefore cannot meaningfully predict the future.

As a concrete example, consider foraging. Foraging is informally defined as the sequential search for resources (Barack, 2024). It is a central behavior of all mobile agents, be they living organisms, AI bots, or robots. Standard formalizations of foraging have significant limitations in terms of characterizing real-world behaviors. Most importantly for this work, classical models of foraging assume that the environment is essentially stationary. For example, without the single agent's intervention, the resources stay available indefinitely. This is a poor model of real-world foraging, because typically resources are depleted, rather than persistent.

The rest of the paper is organized as follows: Section 2 summarizes the prospective learning framework and the main results from (De Silva et al., 2024) in the setting where decisions don't affect

the future outcomes. Section 3 formalizes prospective learning with control. Section 4 takes a step to theoretical results, showing the existence of a sequence of hypotheses whose risks converge to the Bayes optimal risk. Section 5 introduces the Prospective Foraging (ProForg). Section 6 demonstrates that ProForg can prospectively learn to optimally control future outcomes.

## 2 Prospective Learning without Control

Here we briefly review our prior work on this topic, which we previously called "prospective learning" (De Silva et al., 2023; 2024; Bai et al., 2026), modifying notation slightly for convenience. In retrospect, it would more specifically be called prospective learning without control.

**Data** Let the input and output be denoted by $x_t \in \mathcal{X}$ and $y_t \in \mathcal{Y}$, respectively. Let $z_t = (x_t, y_t)$. We will denote the observed data, $z_{\leq t} = \{z_1, \dots, z_t\}$, and the unobserved data, $z_{>t}$. In contrast to PAC learning, $t$ is not just a dummy variable, but rather, indexes time. Let $t \in \mathcal{T} \subseteq \mathbb{N}$, where $\mathbb{N} = \{1, 2, \dots\}$ are the counting numbers, and let $|\mathcal{T}| = T \leq \infty$ be the total number of time steps. We therefore define the data triple $(x_t, y_t, t)$.

**Hypothesis** The hypothesis space generalizes the notion from PAC learning. Here, $h_t : \mathcal{X} \times \mathcal{T} \to \mathcal{Y}$ is a hypothesis in $\mathcal{H}_t \subseteq \{h : \mathcal{X} \times \mathcal{T} \to \mathcal{Y}\}$, so $h_t$ infers an output $y_{t+1}$ for a given input $x_t$ at time $t$.Denote the hypothesis sequence, $h \equiv (h_1, h_2, \dots)$, where each element of this sequence $h_t : \mathcal{X} \mapsto \mathcal{Y}$. At times we will consider a sequence of nested hypothesis spaces, $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \cdots \subseteq \mathcal{H}$.

**Loss** The instantaneous loss, $\bar{\ell} : \mathcal{H} \times \mathbb{Z} \to \mathbb{R}$, is a map from the hypothesis and current data to the reals. In an abuse of notation we write $\bar{\ell}(h_t, z_t) \equiv \bar{\ell}(h(x_t, t), y_t)$. In all real-world problems, we care about the integrated future loss. Let $w_t$ be a non-increasing non-negative weighting function that sums to one, that is $\sum_t w_t = 1$ and $0 \leq w_t \leq 1 \, \forall i$. We thus define loss at time $t$ as $\ell_t(h, z_{>t}) = \sum_{s>t} w_{s-t} \bar{\ell}(h(x_s, s), y_s)$, which is the weighted cumulative loss over all the future data.[1,2] To fully specify loss therefore requires specifying a temporal discounting function, $\boldsymbol{w} \equiv (w_1, w_2, \dots)$. For example, we could let $\boldsymbol{w}$ be an exponentially decaying function, $w_{s-t} = \gamma^{s-t}$, where $\gamma \leq 1$.[3]

**Learner** Let a learner, $\mathcal{L}$ map from the data to a hypothesis, $\mathcal{L}(\mathcal{D}_t) = \hat{h}^t$, where $\mathcal{D}_t = (x_s, y_s, s)_{s \leq t}$, and $\hat{h}^t$ is the estimated hypothesis sequence obtained by the learner at time $t$.[4]

**Assumptions** Let $\boldsymbol{X} = X_{0<t}$ and $\boldsymbol{Y} = Y_{0<t}$ be stochastic processes, taking values $\boldsymbol{X} = x_{0<t}$ and $\boldsymbol{Y} = y_{0<t}$, respectively. Assume $(\boldsymbol{X}, \boldsymbol{Y}) \sim F \in \mathcal{F}$. In general, we place no assumptions on $\mathcal{F}$.

**Theoretical Goal** Define risk as expected loss, $R_t(h) = \mathbb{E}_F[\ell_t(h, Z) \mid z_{\leq t}] = \int \ell_t(h, Z) d\mathbb{P}_{Z|z_{\leq t}}$.[5] Let $R_t^* = \min_{h \in \mathcal{H}_t} R_t(h)$ be the (Bayes) optimal risk.[6]

We seek a learner, $\mathcal{L}$ with the following property: for any $F \in \mathcal{F}$, and $\epsilon, \delta > 0$, there exists a $t'(\epsilon, \delta)$ such that the learner outputs a hypothesis $\hat{h}^t$ satisfying $\mathbb{P}[R_t(\hat{h}^t) - R_t^* < \epsilon] \geq 1 - \delta$, for any $t > t'(\epsilon, \delta)$.

One implication of the above desiderata is that the estimated hypothesis, $\hat{h}^t$, is a consistent estimator of the risk (Shalev-Shwartz and Ben-David, 2014). In other words, $R(\hat{h}^t) \to R_t^*$ as $t \to \infty$. The above goal, however, is not an explicit objective function. In practice, therefore, we specify an objective function to minimize. The hope is that if we minimize this objective, then eventually, the resulting estimand satisfies the above desiderata. We seek to minimize the weighted sum of

[1] If $T$ is infinite, then we replace the sums with limiting sums, e.g., $\lim_{T \to \infty} \sum_{t=1}^{T} w_t$, to ensure that the sum converges.

[2] Note that if we let $w_t$ be a constant function of $t$, we recover something equivalent to the loss in PAC learning.

[3] This is a slight variation of how prospective loss was previously defined in De Silva et al. (2024).

[4] Formally, we assume that $h$ is a random variable and $h \in \sigma(Z_{\leq t})$ where $\sigma(\cdot)$ denotes the filtration (an increasing sequence of sigma algebras) of the stochastic process $Z$.

[5] This is a slight abuse of notation, because previously $\ell$ mapped from a hypothesis and data corpus, but now we are saying that it maps from a hypothesis and a collection of random variables. We have used the shorthand $\mathbb{E}[Y \mid x]$ for $\mathbb{E}[Y \mid X = x]$. Note that $\mathbb{E}[Z|z_{\leq t}]$ is equivalent to $\mathbb{E}[Z_{>t}|z_{\leq t}]$ because the past $z$'s are given.

[6] In a slight abuse of notation, we use $\min$ in place of $\inf$ even though the $\min$ might not exist. Here we use the shorthand $\mathcal{H}_t \equiv \sigma(Z_{\leq t})$.

instantaneous losses on *future* data:

$$\hat{h}^t = \operatorname*{argmin}_{h \in \mathcal{H}_t} R_t(h) = \operatorname*{argmin}_{h \in \mathcal{H}_t} \int \ell_t(h, Z) d\mathbb{P}_{Z|z_{\leq t}} = \operatorname*{argmin}_{h \in \mathcal{H}_t} \int \sum_{s>t} w_{s-t} \bar{\ell}(h(x_s, s), y_s) d\mathbb{P}_{Z|z_{\leq t}}. \tag{1}$$

If $z$ is a discrete set, the above integral becomes a sum. Let $p_z$ denote the probability of a particular sequence $z \in \mathbb{Z}$. With this notation, Equation (1) becomes

$$\hat{h}^t = \operatorname*{argmin}_{h \in \mathcal{H}_t} \sum_{z \in \mathbb{Z}: z_{\leq t}} p_z \ell_t(h, Z) = \operatorname*{argmin}_{h \in \mathcal{H}_t} \sum_{z \in \mathbb{Z}: z_{\leq t}} p_z \sum_{s>t} w_{s-t} \bar{\ell}(h(x_s, s), y_s). \tag{2}$$

**Empirical Goal** In practice, in general, we cannot minimize Equation (1) because it requires solving an intractable integral, and we typically cannot solve Equation (2) because it requires sampling too many possible futures (there are exponentially many even if $z$ is binary). Moreover, we cannot minimize Equations (1) and (2) because they both require sampling from an unknown distribution, $\mathbb{P}_{Z|z_{\leq t}}$. To address the first issue, we use a Monte Carlo approximate sum, sampling only a subset of possible futures. To address the second issue, rather than sampling from the unobserved future, we sample only from the observed past. With these two approximations in hand, we have

$$\hat{h}^t \approx \operatorname*{argmin}_{h^t \in \mathcal{H}_t} \sum_{t'>0}^{t} \ell_{t'}(h, z_{>t'}) \approx \operatorname*{argmin}_{h^t \in \mathcal{H}_t} \sum_{t'>0}^{t} \sum_{s>t'}^{t} w_{s-t'} \bar{\ell}(h^{t'}(x_s, s), y_s). \tag{3}$$

**Main result** The main result of De Silva et al. (2024) is approximating a variant of Equation (1), under suitable assumptions, satisfies our theoretical goal. The key assumptions are (a) consistency and (b) uniform concentration. The focus in that work was on settings in which our decisions $\hat{h}^t(x_s, s)$ had no impact on future outcomes or distributions. In such cases, we can ignore the sum in Equation (1), because all that matters for the current decision is the instantaneous loss. Here, we are interested in general settings where the decisions also impact future distributions and losses.

## 3 Prospective Learning with Control

We modify the above prospective learning framework to be able to accommodate stochastic decision processes. In other words, we generalize from learning to infer, to learning to control. The key difference is in the loss. For inference, the loss is the error relative to the truth/optimal, which is always observed. In control, we do not necessarily see the truth/optimal, so one cannot define loss as an error term directly. Below, we formalize this distinction.

**Data** Let $x_t$ be the position of the agent in an environment at time $t$. In reinforcement learning (RL), this would often be called the "observation state." Thus, if the environment is two-dimensional, finite, rectangular, and discrete, then it has width $W$ and height $H$, and $x_t \in \mathcal{X} \equiv [W] \times [H]$ is the position of the agent at time $t$. Then $y_t$ is the reward the agent would receive at time $t$ at any of the locations, so $y_t \in \mathcal{Y} \equiv \mathbb{R}_+^{W \times H}$. As before, we have time $t \in \mathcal{T} \subset \mathbb{N}$ as part of the data triple at each time step.

**Hypothesis** The hypothesis maps from the current position and time to the next position, $h : \mathcal{X} \times \mathcal{T} \to \mathcal{X}$. Without an estimate of the rewards, however, we have no way, in general, to estimate what the rewards would be if we changed the hypothesis. Thus, we could either let the hypothesis map to the next position and the next reward, $h : \mathcal{X} \times \mathcal{T} \to \mathcal{X} \times \mathbb{R}_+$, and so $h(x_s, s) \to x_{s+1}, y_{s+1}(x_{s+1})$. More generally, the hypothesis maps to the next position and the reward at all possible locations, $h : \mathcal{X} \times \mathcal{T} \to \mathcal{X} \times \mathbb{R}_+^{W \times H}$, so $h(x_s, s) \to x_{s+1}, y_{s+1}$.

**Loss** The instantaneous loss operates on the current hypothesis and data, $\bar{\ell} : \mathcal{H} \times \mathbb{Z} \to \mathbb{R}_+$. The instantaneous loss is the negative reward the agent receives at the position the agent is in at that time. Recall that $x_s$ is a two-dimensional vector encoding the position of the agent at time $s$, and that $y_{s+1}$ encodes the reward at all locations at time $s+1$, so, in a slight abuse of notation, we can write

$$\bar{\ell}(h_s(x_s, s), y_{s+1}) = -y_{s+1}(x_{s+1}). \tag{4}$$

Above, we use the notation $y_s(x_s)$ to indicate the reward specifically at location $x_s$, rather than all possible locations. The loss at time $t$ is the cumulative instantaneous loss, $\sum_{s>t} w_{s-t} \bar{\ell}(h_s(x_s, s), y_{s+1})$.

**Learner** The learner $\mathcal{L}$ is a map from the observed data, $\mathcal{D}_t$ to a hypothesis, $\hat{h}^t$. The observed data are $\mathcal{D}_t = (x_s, y_{s+1}(x_{s+1}), s)_{s \leq t-1}$.

**Assumptions** Let $\boldsymbol{X} = X_{0,1,2,\ldots}$ and $\boldsymbol{Y} = Y_{0,1,2,\ldots}$ be stochastic decision processes, that is, $(\boldsymbol{X}, \boldsymbol{Y}) \sim F \in \mathcal{F}$, where $Y_{>t}$ can be an arbitrary function of $(X_{\leq t}, Y_{\leq t})$. Denote the values that the random variables $\boldsymbol{X}$ and $\boldsymbol{Y}$ as $\boldsymbol{x}$ and $\boldsymbol{y}$, respectively. Note that the distribution of $X_t$ is, in general, a function of the previous sequence of observed $x_{\leq t}$, which is itself a function of the observed rewards $y_{\leq t}$, and the estimated hypotheses, $\hat{h}^{\leq t}$. If we assume that the learners and hypotheses are deterministic functions of the data, then future distributions of $X_s$ are functions of the learner and hypotheses, but not conditionally independent on them, once conditioned on the data.

**Theoretical Goal** The goal of learning a hypothesis whose risk converges to Bayes optimal risk remains the same in PF as in PL. However, the objective function we seek to optimize is slightly modified, because PF is inherently sequential. Thus, the instantaneous loss in PL from Equation (1) is replaced with the instantaneous loss in PF from Equation (4):

$$\hat{h}^t = \operatorname*{argmin}_{h \in \mathcal{H}_t} R_t(h) \equiv \operatorname*{argmax}_{h \in \mathcal{H}_t} \int \sum_{s>t} w_{s-t} \times y_{s+1}(x_{s+1}) d\mathbb{P}_{Z|z_{\leq t}} = \operatorname*{argmax}_{h \in \mathcal{H}_t} \sum_{z \in \mathbb{Z}: z_{\leq t}} p_z \sum_{s>t} w_{s-t} \times y_{s+1}(x_{s+1}). \tag{5}$$

Note that while the argmax is with respect to $h$, on the right hand side after the "$\equiv$" the $h$ is implicit. We could notationally let $y_s$ be a function of $h$ to clarify if we desired.

**Empirical Goal** Much like PL, in PF, we cannot directly solve Equation (5) because we cannot sample all possible futures, and we do not know the likelihood of each possible future. Thus, we modify the sums in Equation (5) to only sum up to the current time, $t$:

$$\hat{h}^t \approx \operatorname*{argmin}_{h^t \in \mathcal{H}_t} \sum_{t'>0}^{t} \sum_{s>t'}^{t} -w_{s-t'} \times y_{s+1}(x_{s+1}). \tag{6}$$

**Relationship to prospective learning without control** The observed *data* triples are now $(x_s, y_{s+1}(x_{s+1}), s)$, rather than $(x_s, y_s, s)$. Note that this differs in two ways from PL. First, the index on $y$ is not $s$ but $s+1$, this is required because the rewards are necessarily a function of the action of the agent. Second, rather than the entire vector of output being observed as in PL, in PF we only obtain a partial observation, the reward at the location we happen to currently inhabit. This renders PF closely related to causal inference, in which the outcomes are defined as a function of the intervention (or treatment), and therefore only partially observed, leading to the potential outcomes framework for learning and inference. *Hypotheses* are no longer maps from $\mathcal{X} \times \mathcal{T}$ to $\mathcal{Y}$, rather, they map to $\mathcal{X}$ and $\mathbb{R}_+$. *Loss* in PF is the cumulative instantaneous loss, as in PL. However, in PL, instantaneous loss is typically an error signal, e.g., the difference between the predicted output and the actual output. In PF, loss is just negative reward. Moreover, loss is not observed for any of the decisions that the agent did not make, necessitating the use of some surrogate loss function in practice. *Learners* map from data to hypotheses, though the data are subtly different due to the time indices and partial observation. *Assumptions* can be the same, though note that the future data are now necessarily a function of the learner and hypothesis, whereas they were not necessarily in prospective learning. The *theoretical goal* is the same, modulo replacing the loss from PL with that from PF. The *empirical goal* changes somewhat similarly, in so far that in PF we cannot directly solve Equation (5), so we use a surrogate optimization problem, Equation (6).

## 4 Theoretical Results

Here we provide proof sketches, formal proofs are in Appendix B.

**Lemma 4.1.** *Suppose $\{Z_t\}_{t=1}^{\infty}$ is a stochastic process and let $\{\mathcal{H}_t\}_{t=1}^{\infty}$ be an increasing hypothesis class, such that there exists $h^{(t)} \in \mathcal{H}_t$ which satisfies*

$$\lim_{t \to \infty} \mathbb{E}[R_t(h^{(t)}) - R_t^*] = 0. \tag{7}$$

*Additionally, suppose* $\{u_t\}_{t=1}^{\infty}$ *is a sequence such that* $u_t \to \infty$ *as* $t \to \infty$*, and* $u_t \leq t$ *for all* $t$*. Define the partial cumulative prospective loss to be*

$$e_m(h) = \sum_{s=1}^{m} w_s \tilde{y}_{s+1}(x_{s+1}).$$

*Then, there exists* $\tilde{h}^{(t)} \in \mathcal{H}_t$ *such that*

$$\lim_{t\to\infty} \mathbb{E}\left[\sup_{u_t \leq m \leq \infty} e_m(\tilde{h}^{(t)}) | Z_{\leq t}\right] - R_t^* = 0.$$

**Sketch of proof.** A condition for the above lemma to hold is the existence of a sequence of hypothesis whose risks converge to the Bayes' risk. We establish that for this asymptotically optimal sequence of hypotheses, there exists a subsequence of timepoints on which the expected partial cumulative loss is arbitrarily close to the Bayes' risk, using Markov's Inequality and Borel-Cantelli Lemma. We use this subsequence of timepoints to establish that for a particular sequence of hypotheses, the expected partial cumulative loss approaches the Bayes' risk.

**Theorem 4.1.** *Consider a family of stochastic processes* $(\mathbf{X}_t, \mathbf{Y}_t)_{t>0}$*. Suppose there exists a hypothesis class such that* $\mathcal{H}_1 \subseteq \mathcal{H}_2 \ldots$ *such that*

$$\lim_{t\to\infty} \left[\inf_{h\in\mathcal{H}_t} R_t(h) - R_t^*\right] = 0.$$

*Also, there exist sequences* $\{u_t\}_{t=1}^{\infty}$ *and* $\{\xi_t\}_{t=1}^{\infty}$ *satisfying* $\lim_{t\to\infty} \xi_t = 0$*, and* $u_t \leq t$*, such that*

$$\mathbb{E}\left[\max_{h\in\mathcal{H}_t} \left| \sum_{s=t+1}^{\infty} w_{s-t} \tilde{y}_{s+1}(x_{s+1}) - \max_{u_t \leq m \leq t} \sum_{s=1}^{m} w_s \tilde{y}_{s+1}(x_{s+1}) \right| \right] \leq \xi_t.$$

*Then, there exists a sequence* $\{i_t\}_{t=1}^{\infty}$ *such that*

$$\hat{h}^{(t)} = \operatorname*{argmin}_{h\in\mathcal{H}_{i_t}} \max_{u_{i_t} \leq m \leq t} \sum_{s=1}^{m} w_s \tilde{y}_{s+1}(x_{s+1})$$

*reaches the Bayes' optimal risk.*

**Sketch of proof.** We note that for any sequence of hypotheses, the difference between the prospective loss and the partial cumulative loss is bounded by terms approaching zero. We find a subsequence of timepoints on which this difference is sufficiently small, and using Lemma 4.1 we establish that the Bayes' risk of the minimzer of the partial cumulative loss on that particular subsequence of timepoints, approaches zero.

## 5 Estimation and inference for prospective control

### 5.1 Estimation

**Cumulative loss estimation** The empirical goal in prospective control, Equation (6):

$$\hat{h}^t \approx \operatorname*{argmin}_{h^t\in\mathcal{H}_t} \sum_{t'>0}^{t} \sum_{s>t'}^{t} -w_{s-t'} \times y_{s+1}(x_{s+1}).$$

Based on the above, a direct way is we build a data corpus using prospective loss as follows. Let $X_t = (\widetilde{x}_s, \widetilde{s})_{s\leq t}$ be the concatenation of the embedded positions and times, such that $X_t \in \mathbb{R}^{d\times t}$, where $d$ is the total dimension of encoded states and time. Let $Y_t = (\sum_{t'=s+1}^{t} -w_{s-t'} \times y_t(x_{t'}))_{s\leq t}$, such that $Y_t \in \mathbb{R}^t$. Then, we can feed those data into any supervised learning machine algorithm, for example, a decision forest or deep network. Let the resulting regression function be denoted, $\hat{g}^p$, which maps from a state, $(x_s, s)$ to an estimated prospective loss $\bar{y}_s \in \mathbb{R}_+$.

**Instantaneous loss estimation** When searching over hypotheses, some will result in positions at various times that are not identical (i.e., *counterfactual*) to the positions that were selected by the estimated hypothesis at that time, $\hat{h}^s$. We therefore lack observations of $y_s$ for all the counterfactual positions, and in practice, we cannot actually solve the above optimization problem, because we have missing data. Instead, we introduce a further approximation, replacing $y_{s+1}$ with $\tilde{y}_{s+1}$. Define $\widetilde{y}_s$ as follows. Let $x'_s$ denote *potential outcomes* that the agent can take at time $s$, whereas $x_s$ is the position that the agent actually took at that time, and let $\hat{y}_s(x'_s)$ denote the estimated reward at time $s$ at position $x'_s$ based on the current hypothesis:

$$\widetilde{y}_s(x_s) = \begin{cases} y_s(x_s) & x'_s = x_s \\ \hat{y}_s(x'_s) & x'_s \neq x_s. \end{cases} \qquad \triangleright \text{ these are counterfactuals} \tag{8}$$

Putting all this together yields:

$$\hat{h}^t \approx \underset{h^t \in \mathcal{H}_t}{\operatorname{argmin}} \sum_{t'>0}^{t} \sum_{s>t'}^{t} -w_{s-t'} \times \tilde{y}_{s+1}(x_{s+1}). \tag{9}$$

Since we are estimating $y$ at this point, we are no longer restricted to summing up until time $t$ for the inner sum. Let $T$ be some predefined horizon, we have

$$\hat{h}^t \approx \underset{h^t \in \mathcal{H}_t}{\operatorname{argmin}} \sum_{t'>0}^{t} \sum_{s>t'}^{t+T} -w_{s-t'} \times \tilde{y}_{s+1}(x_{s+1}). \tag{10}$$

In this case, if we desire to use instantaneous loss, we can build a data corpus as follows. $X_t$ remains the same as it above. However, $Y_t = (y_{s+1}(x_{s+1}))_{s \leq t}$, which is also in $\mathbb{R}^t$. Let the resulting regression function be denoted $\hat{g}^i$, which maps from a state $(x_s, s)$ to an estimated instantaneous loss $\hat{y}_I \in \mathbb{R}_+$.[7]

### 5.2 Inference

Recall that our goal is Equation (6). And now, we get estimated instantaneous reward, $\tilde{y}_s$, and cumulative reward $\bar{y}_{t+T}$ receptively. One effective strategy is to combine this two parts to approximate the prospective loss. This yields both counterfactual insight and a tractable approximation to future loss. To be more specific, given a predefined horizon, we have

$$\hat{h}^t \approx \underset{h^t \in \mathcal{H}_t}{\operatorname{argmin}} \sum_{t'>0}^{t} \{w_{t+T-t'} \times \bar{y}_{t+T} + \sum_{s>t'}^{t+T} -w_{s-t'} \times \tilde{y}_{s+1}(x_{s+1})\},$$

where $\bar{y}_{t+T}$ represents a estimation of terminal cost at time $t+T$, and $tildey_s$ is the a estimated instantaneous cost at time $s$.

**Inference in Practice** When we are running ProForg, we evaluate each trajectory via calculating its prospective loss, $-\{w_T \times \hat{g}^p(x_{s+T}, s+T) + \sum_{j=s}^{s+T} w_{j-s} \times \hat{g}^i(x_s, s)\}$. Assuming $T = 6$, and $A = 2$, that means we have $2^6 = 64$ different trajectories, each with an associated prospective loss, as estimated by iterating the instantaneous losses and adding the prospective tail at the end. We select the daughter node which has the trajectory with the maximum. In practice, we deploy ProForg online. That is, for each time $t$, the decision is made by learner trained on past observations till time $t-1$ . Algorithm 1 provides details. The above is an exhaustive search strategy. If our computational budget is inadequate to search all the possible trajectories, then it is wise to efficiently search a subset of possible trajectories. A common and relatively simple strategy for that is a Monte Carlo Tree Search (MCTS).

## 6 Experimental Validation

To validate the above described theoretical framework, we devise a specific prospective learning with control problem, namely, prospective foraging.

[7] We implicitly assumed in the above notation that $\hat{g}$ implements the embedding function.

### 6.1 PROSPECTIVE FORAGING

Informally, we assume an agent is navigating an environment, and resources (also called affordances (Gibson, 2014) or rewards (Sutton and Barto, 2018)) appear and disappear in that environment in various locations. At any given time, the agent may move its position, and the loss is the negative reward associated with where the agent is.

**Data** Assume that $\mathcal{X}$ is a discrete two-dimensional grid, and let $H = 1$, so the environment is a chain (Figure 1. Thus, $x_s \in \{1, \ldots, W\}$, and $y_s \in \mathbb{R}_+^W$. We potentially encode position via a one-hot encoding, thereby $x_s$ becomes a binary $W$-dimensional vector. We also embed time into a 50-dimensional vector using the classical 'positional' encoding strategy, as we have before in De Silva et al. (2024). Let $\tilde{x}_s$ and $\tilde{s}$ represent the embedded position and time.

**Hypothesis** At each time step, the hypothesis can move the agent by one cell in the grid in either direction assuming it is not at a boundary, or stay put. At a boundary, it can only move nowhere or in the opposite direction. Formally, $h(x_s, s) \to x_{s+1} \in \mathcal{X}_s$, where $\mathcal{X}_s$ could be either $\{x_s - 1, x_s, x_s + 1\}$, $\{x_s - 1, x_s\}$, or $\{x_s, x_s + 1\}$.

**Loss** The loss is defined as above, with the temporal discounting factor of $\gamma$.

**Assumptions** Let $F_{\boldsymbol{Y}}$ denote the prior distribution of $\boldsymbol{Y}$. Assume that $\boldsymbol{Y}$ is independent of $\boldsymbol{X}$, that is, $F_{\boldsymbol{Y}|\boldsymbol{X}} = F_{\boldsymbol{Y}}$. Let $y_s \in \mathbb{R}_+^W$ be a $W$-dimensional non-negative vector, where each element encodes the reward the agent would receive at location $x_s$ at time $s$. Assume that for two locations, $A$ and $B$, there are rewards sometimes, but for the other $W-2$ locations, there are never any rewards. For locations $A$ and $B$, assume that rewards arrive on a schedule. Informally, at every $r$ time steps, location $A$ receives a boost of one reward, which then decays exponentially down to zero at a rate of $\tau$. Location $B$ has the same temporal dynamics, but shifted by $r/2$ time steps. Thus,

$$y_s(A) = e^{-(s \mod r)/\tau}, \qquad y_s(B) = e^{-((s+r/2) \mod r)/\tau},$$

where $s \mod r$ indicates the modulus function. The dynamics on the conditional distribution, $F_{\boldsymbol{X}|\boldsymbol{Y}}$ are a function of the learned hypotheses, $\hat{h}$.

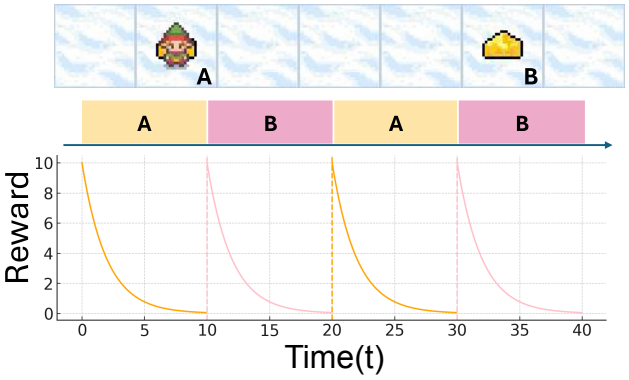


Figure 1: 1D foraging environment.

We implement and evaluate the prospective forager (ProForg) within the 1-D foraging environment described in Section 6.1 and (Bai et al., 2026): the agent forages along a $1 \times 7$ linear track containing only two reward patches, A and B, located three grid cells apart, as shown in Figure 1. Every $r = 10$ timestamps, rewards alternatively appear at A and B. The agent's goal is to maximize the total reward in its single lifetime, meaning there are no resets within each run.

To implement ProForg in practice, inspired by positional encoding in Transformer (Vaswani et al., 2017), ProForg incorporates the same time-embedding that (De Silva et al., 2024) have used:

$$\varphi_f(t) = (\sin(\omega_1 t), \ldots, \sin(\omega_{d/2} t), \cos(\omega_1 t), \ldots, \cos(\omega_{d/2} t)), \tag{11}$$

where $\omega_i = \pi/i$ for $i = 1, \ldots, d/2$.

By design, both the reward function and the optimal policy are known for this environment. Therefore, we report the normalized prospective regret which is defined to be the average difference

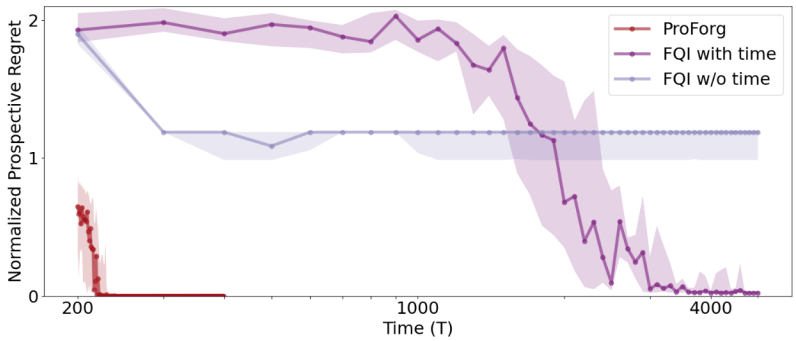


Figure 2: **ProForg efficiently achieves Bayes optimal regret.** Normalized prospective regret of ProForg (red), time-aware Fitted Q-Iteration (FQI with time, purple, our invention to improve FQI) and Time-agnostic Fitted Q-Iteration(FQI w/o time, lavender, the published algorithm) Ernst et al. (2005). While both ProForg and time-aware FQI converge to having zero-regret, ProForg is orders of magnitude more efficient than the latter.

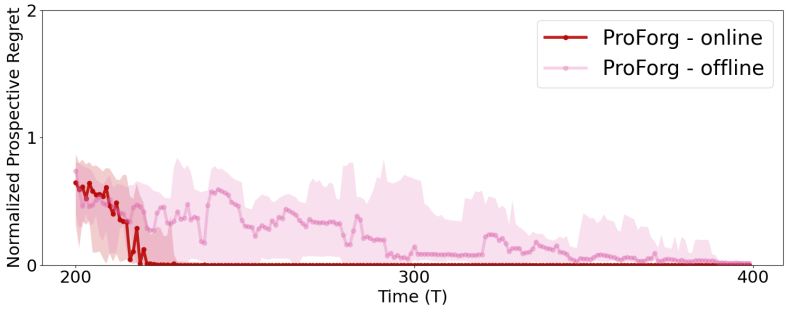


Figure 3: **ProForg online is more efficient than offline.** Normalized prospective regret for ProForg: online (red) and offline (pink). After pre-training the online one for 200 time steps, it converges in another 50 time steps, whereas the offline one requires about 4x more data to converge.

between prospective losses of the agent and the optimal policy for a predefined horizon $T$:

$$\Delta\ell(x_s, \ldots, x_{s+T}) = \frac{1}{T}\sum_{i=s}^{s+T} w_{i-s}(y_s(x_s^*) - y_s(x_s)), \tag{12}$$

where $(x_s^*, x*_{s+1}, ...., x_{s+T}^*)$ is the states returned by optimal policy at corresponding time.

### 6.2 Numerical Results

**ProForg** After an initial warm-up phase, we let the ProForg learn in an online manner as it interacts with the environment over time (Figure 2). We detail the steps of this process in Algorithm 1(see Appendix A). We compute the normalized prospective regret of ProForg over time, and compare to Fitted Q-Iteration (FQI) (Ernst et al., 2005; Munos and Szepesvari, 2008), a classical reinforcement learning algorithm, and its time-aware variant that we introduce here to enable it to work in non-stationary environments (see Appendix C and algorithm 2). In both ProForg and FQI variants, we use Random Forests as our chosen class of regressors. We find that ProForg converges to zero-regret significantly faster than the time-aware FQI, and that the time-agnostic converges to a sub-optimal regret regardless of the time spent interacting with the environment.

**Online or Offline?** Building on the online formulation, we compare the online and offline ProForg, under the same environment settings and parameters for training and inference (Figure 3). The only difference is the next state selection: online-ProForg chooses the next step using its currently learned policy, while offline-ProForg explores the next step uniformly at random.

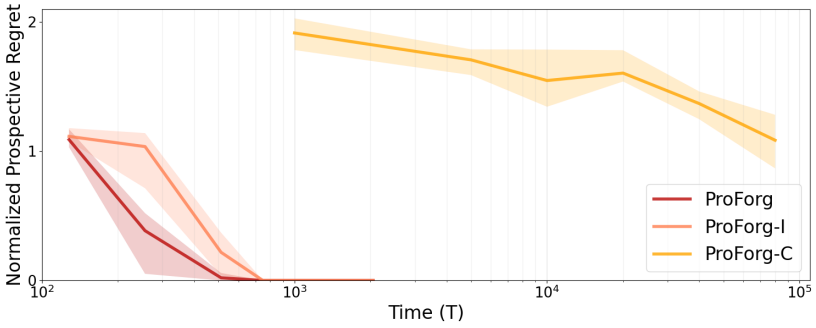


Figure 4: Normalized prospective regret for ProForg(dark red), ProForg-I(orange) and ProForg-C (yellow). Removing either component reduces performance relative to ProForg.

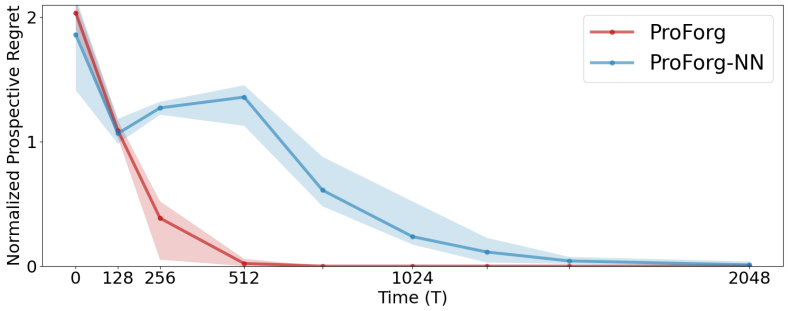


Figure 5: **ProForg with decision forests is more efficient than with neural networks.** Normalized prospective regret for ProForg with Gradient-Boosted Trees (red) and MLP Regressor (blue). While ProForg is more efficient, ProForg-NN does converge as well.

**Instantaneous, Cumulative or Both?** ProForg employs two learners during the training and inference phases. $\mathrm{ML_I}$, fit to instantaneous loss, and $\mathrm{ML_C}$ fit to cumulative (future) loss. During inference, we combine a finite-horizon "lookahead" from $\mathrm{ML_I}$ with a terminal cost approximation from $\mathrm{ML_C}$. We compare offline invariants of Instantaneous-only (ProForg-I), Cumulative-only (ProForg-C), and combined (ProForg) (Figure 4). Using the terminal cost alone (ProForg-C) converges quite inefficiently, but we find that adding it to the instantaneous loss accelerates learning.

**ProForg with Neural Network** In Figure 5, we show that neural networks can also be used as the regressor in ProForg. While the ProForg with a neural network (specifically, a multi-layer perceptron with two hidden layers comprising of 128 units) converges eventually, the convergence is slower than the ProForg employing a random forest as the regressor.

## 7 Discussion

In this work, we introduce the framework of prospective control. Prospective Foraging, a concrete example of prospective control, learns to optimally behave in the future in a dynamic environment. Our results show that ProForg can plan ahead and decide its next step guiding it towards optimality. We also observe that standard RL baselines fail to converge to the optimal policy despite the simple nature of the 1-D foraging environment. Though much work is to be done, we prospective control is an initial step towards understanding and utilizing optimal control of the future in both natural and artificial intelligences. We leave generalizing to continuous action, state and time spaces, and deploying our algorithms in complex real-world environments as future work.

## REFERENCES AND NOTES


Ashwin De Silva, Rahul Ramesh, Lyle Ungar, Marshall Hussain Shuler, Noah J Cowan, Michael Platt, Chen Li, Leyla Isik, Seung-Eon Roh, Adam Charles, Archana Venkataraman, Brian Caffo, Javier J How, Justus M Kebschull, John W Krakauer, Maxim Bichuch, Kaleab Alemayehu Kinfu, Eva Yezerets, Dinesh Jayaraman, Jong M Shin, Soledad Villar, Ian Phillips, Carey E Priebe, Thomas Hartung, Michael I Miller, Jayanta Dey, Ningyuan Huang, Eric Eaton, Ralph Etienne-Cummings, Elizabeth L Ogburn, Randal Burns, Onyema Osuagwu, Brett Mensh, Alysson R Muotri, Julia Brown, Chris White, Weiwei Yang, Andrei A Rusu Timothy Verstynen, Konrad P Kording, Pratik Chaudhari, and Joshua T Vogelstein. Prospective Learning: Principled Extrapolation to the Future. In Sarath Chandar, Razvan Pascanu, Hanie Sedghi, and Doina Precup, editors, *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 347–357. PMLR, 2023. 1, 2

Ashwin De Silva, Rahul Ramesh, Rubing Yang, Joshua T Vogelstein, and Pratik Chaudhari. Prospective Learning: Learning for a Dynamic Future. 2024. 1, 2, 3, 7

Yuxin Bai, Cecelia Shuai, Ashwin De Silva, Siyu Yu, Pratik Chaudhari, and Joshua T Vogelstein. *Prospective learning in retrospect*, pages 17–29. Lecture notes in computer science. Springer Nature Switzerland, 2026. 1, 2, 7

Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning (Adaptive Computation and Machine Learning series)*. The MIT Press, kindle edition, 2012. 1

Dimitri Bertsekas. *A course in Reinforcement Learning*. Athena Scientific, 2023. 1

Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018. 1, 7

Alexander L. Strehl, Lihong Li, and Michael L. Littman. Reinforcement learning in finite mdps: Pac analysis. *Journal of Machine Learning Research*, 10:2413–2444, 2009. 1

David Silver, Aja Huang, Chris J. Maddison, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. 1

Annie S Chen, Archit Sharma, Sergey Levine, and Chelsea Finn. You Only Live Once: Single-Life Reinforcement Learning via Learned Reward Shaping. In *Decision Awareness in Reinforcement Learning Workshop at ICML 2022*, 2022. 1

Shai Shalev-Shwartz. Online learning and online convex optimization. *Foundations and Trends in Machine Learning*, 4(2):107–194, 2012. 1

David L Barack. What is foraging? *Biology & Philosophy*, 39:3, 2024. 1

Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014. 2

James J Gibson. *The Ecological Approach to Visual Perception: Classic Edition (Psychology Press & Routledge Classic Editions)*. Psychology Press, 1 edition, 2014. 7

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. 7

Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6, 2005. 8, 14

Remi Munos and Csaba Szepesvari. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9(5), 2008. 8, 14

# A ALGORITHM

**Algorithm 1** ProForg: Our main algorithm for learning to control in non-stationary environments.

**Input:** Sequence $\{(x_s, s, y_s)\}_{s=1}^{T}$, Online update timestamps $T'$, discount $\gamma \in (0,1)$, horizon $H$, $\mathrm{embed}(\cdot)$

**Initialize regressor** ML

**for** $s = 1$ **to** $T$ **do**
  $x'_s \leftarrow [\, x_s,\ \mathrm{embed}(s)\,]$
  $y'_s \leftarrow \sum_{t'=s+1}^{T} \gamma^{\,t'-(s+1)}\, y_{t'}$ ▷ discounted future return starting at $s{+}1$
**end for**
Train supervised learning regressor $\mathrm{ML_I}$ on $\{(x'_s, y_s)\}_{s=1}^{T}$
Train supervised learning regressor $\mathrm{ML_C}$ on $\{(x'_s, y'_s)\}_{s=1}^{T}$

**Online update regressor** ML
$x_{t'} \leftarrow x_T$
**for** $t' = 1$ **to** $T'$ **do**
  $\mathcal{C} \leftarrow \textsc{AllPaths}(x_{\text{state}}, H)$ ▷ each $c \in \mathcal{C}$ is a length-$H$ continuation, $c = (x^{(1)}, \ldots, x^{(H)})$
  **for** $c \in \mathcal{C}$ **do**
    $\hat{G}(c) \leftarrow \sum_{h=1}^{H} \gamma^{\,h-1}\, \mathrm{ML_I.predict}\big([\, x^{(h)},\ \mathrm{embed}(t{+}h)\,]\big)$
    $\hat{G}(c) \leftarrow \hat{G}(c) + \gamma^{H} \mathrm{ML_C.predict}\big([\, x^{(H)},\ \mathrm{embed}(t{+}H)\,]\big)$ ▷ approximate future reward
  **end for**
  $c^{\star} \leftarrow \arg\max_{c \in \mathcal{C}} \hat{G}(c)$
  $x_{\text{state}} \leftarrow x^{\star(1)}$ ▷ execute only the first step of the best path
  Update supervised learning regressor $\mathsf{ML_I}$ on $\{(x'_s, y_s)\}_{s=1}^{T+t'}$
  Update supervised learning regressor $\mathsf{ML_C}$ on $\{(x'_s, y'_s)\}_{s=1}^{T+t'}$ ▷ $y's$ here might be different.
**end for**

**return** $\mathsf{ML_I}, \mathsf{ML_C}, \mathcal{S}$ ▷ Final regressor and predicted states

# B THEORETICAL RESULTS

**Lemma B.1.** *Suppose $\{Z_t\}_{t=1}^{\infty}$ is a stochastic process and let $\{\mathcal{H}_t\}_{t=1}^{\infty}$ be an increasing hypothesis class, such that there exists $h^{(t)} \in \mathcal{H}_t$ which satisfies*

$$\lim_{t\to\infty} \mathbb{E}[R_t(h^{(t)}) - R_t^*] = 0. \tag{13}$$

*Additionally, suppose $\{u_t\}_{t=1}^{\infty}$ is a sequence such that $u_t \to \infty$ as $t \to \infty$, and $u_t \le t$ for all $t$. Define*

$$e_m(h) = \sum_{s=1}^{m} w_s \tilde{y}_{s+1}(x_{s+1}).$$

*Then, there exists $\tilde{h}^{(t)} \in \mathcal{H}_t$ such that*

$$\lim_{t\to\infty} \mathbb{E}\left[\sup_{u_t \le m \le \infty} e_m(\tilde{h}^{(t)}) | Z_{\le t}\right] - R_t^* = 0.$$

**Proof.** There exists $h^{(t)} \in \mathcal{H}_t$ such that

$$\lim_{t\to\infty} \mathbb{E}[R_t(h^{(t)}) - R_t^*] = 0.$$

Note that $R_t(h) \ge R_t^*$ almost surely. Choose a subsequence $\{j_k\}_{k=1}^{\infty}$ such that

$$\mathbb{E}[R_{j_k}(h^{(j_k)}) - R_t^*] \le \frac{1}{4^k}.$$

Now,

$$\begin{aligned}\mathbb{E}[R_t(h^{(t)}) - R_t^*] &= \mathbb{E}\left[\mathbb{E}[\lim\sup_{m\to\infty} e_m(h^{(t)})|Z_{\le t}] - R_t^*\right] \\ &= \mathbb{E}\left[\mathbb{E}[\lim_{i\to\infty}\sup_{u_i\le m\le\infty} e_m(h^{(t)})|Z_{\le t}] - R_t^*\right] \\ &= \lim_{i\to\infty}\mathbb{E}\left[\mathbb{E}[\sup_{u_i\le m\le\infty} e_m(h^{(t)})|Z_{\le t}] - R_t^*\right]\end{aligned}$$

Thus, for every $k$, there exists integer $i_k$ such that

$$\mathbb{E}\left[\mathbb{E}[\sup_{u_{i_k}\le m\le\infty} e_m(h^{(j_k)})|Z_{\le j_k}] - R_{j_k}^*\right] \le \mathbb{E}[R_{j_k}(h^{(j_k)}) - R_{j_k}^*] + \frac{1}{4^k} \le \frac{2}{4^k}.$$

Note that,

$$\mathbb{E}\left[\sup_{u_i\le m\le\infty} e_m(h^{(t)})|Z_{\le t}\right] \ge \mathbb{E}\left[\sum_{s=t+1}^{m} w_{s-t}\tilde{y}_{s+1}(x_{s+1})|Z_{\le t}\right] \ge R_t^*.$$

Using Markov's Inequality,

$$\begin{aligned}&\sum_{k=0}^{\infty}\mathbb{P}\left(\mathbb{E}\left[\sup_{u_{i_k}\le m\le\infty} e_m(h^{(j_k)})|Z_{\le j_k}\right] - R_{j_k}^* > 2^{\frac{1}{2}-k}\right) \\ &\le \sum_{k=0}^{\infty}\frac{1}{2^{\frac{1}{2}-k}}\mathbb{E}\left[\mathbb{E}\left[\sup_{u_{i_k}\le m\le\infty} e_m(h^{(j_k)})|Z_{\le j_k}\right] - R_{j_k}^*\right] \\ &\le \sum_{k=0}^{\infty}\frac{1}{2^{\frac{1}{2}-k}}\frac{2}{4^k} \\ &< \infty.\end{aligned}$$

Thus, by Borel Cantelli Lemma, there exists an integer $k_0$ such that for all $k \ge k_0$,

$$\mathbb{E}\left[\sup_{u_{i_k}\le m\le\infty} e_m(h^{(j_k)})|Z_{\le j_k}\right] - R_{j_k}^* \le 2^{\frac{1}{2}-k}.$$

Now, we define $k_t = \max\{k \in \mathbb{N}\cup\{0\} : \max\{i_k, j_k\} \le t\}$. Note that $k_t \to \infty$ as $t \to \infty$, because $i_k, j_k \le \infty$. Define $\alpha_t = 2^{\frac{1}{2}-k_t}$, and observe that $\lim_{t\to\infty}\alpha_t = 0$. Let $t_0 \in \mathbb{N}$ be a random variable such that for all $t \ge t_0$, $k_t \ge k_0$, and thus,

$$\mathbb{E}\left[\sup_{u_{i_{k_t}}\le m\le\infty} e_m(h^{(j_{k_t})})\big|Z_{\le j_{k_t}}\right] - R_{j_{k_t}}^* \le \alpha_t.$$

Choose $\tilde{h}^{(t)} = h^{(j_{k_t})}$ for all $t$. Note that $j_{k_t} \le t \Rightarrow \mathcal{H}_{j_{k_t}} \subset \mathcal{H}_t$. Hence, for $t \ge t_0$,

$$\mathbb{E}\left[\sup_{u_t\le m\le\infty} e_m(\tilde{h}^{(t)})|Z_{\le j_{k_t}}\right] - R_{j_{k_t}}^* \le \mathbb{E}\left[\sup_{u_{i_{k_t}}\le m\le\infty} e_m(\tilde{h}^{(t)})|Z_{\le j_{k_t}}\right] - R_{j_{k_t}}^* \le \alpha_t.$$

Since $\alpha_t \to 0$ as $t \to \infty$, we have,

$$\lim_{t\to\infty}\left(\mathbb{E}\left[\sup_{u_t\le m\le\infty} e_m(\tilde{h}^{(t)})|Z_{\le j_{k_t}}\right] - R_{j_{k_t}}^*\right) = 0.$$

**Theorem B.1.** *Consider a family of stochastic processes $(\mathbf{X}_t, \mathbf{Y}_t)_{t>0}$. Suppose there exists a hypothesis class such that $\mathcal{H}_1 \subseteq \mathcal{H}_2 \ldots$ such that*

$$\lim_{t\to\infty}\left[\inf_{h\in\mathcal{H}_t} R_t(h) - R_t^*\right] = 0.$$

*Also, there exist sequences* $\{u_t\}_{t=1}^{\infty}$ *and* $\{\xi_t\}_{t=1}^{\infty}$ *satisfying* $\lim_{t\to\infty}\xi_t = 0$*, and* $u_t \le t$*, such that*

$$\mathbb{E}\left[\max_{h\in\mathcal{H}_t}\left|\sum_{s=t+1}^{\infty} w_{s-t}\tilde{y}_{s+1}(x_{s+1}) - \max_{u_t\le m\le t}\sum_{s=1}^{m} w_s\tilde{y}_{s+1}(x_{s+1})\right|\right] \le \xi_t.$$

*Then, there exists a sequence* $\{i_t\}_{t=1}^{\infty}$ *such that*

$$\hat{h}^{(t)} = \operatorname*{argmin}_{h\in\mathcal{H}_{i_t}} \max_{u_{i_t}\le m\le t}\sum_{s=1}^{m} w_s\tilde{y}_{s+1}(x_{s+1})$$

*reaches the Bayes' optimal risk.*

**Proof.** We consider a subsequence $\{i_t\}_{t=1}^{\infty}$ such that $\xi_{i_t} \to 0$ exponentially so that $\sum_{t=1}^{\infty}\xi_{i_t} < \infty$. From Markov's Inequality,

$$\begin{aligned}
&\sum_{t=0}^{\infty}\mathbb{P}\left[\max_{h\in\mathcal{H}_{i_t}}\left|\bar{l}_t(h,Z) - \max_{u_{i_t}\le m\le i_t} e_m(h)\right| > \sqrt{\xi_{i_t}}\right] \\
&\le \sum_{t=0}^{\infty}\frac{1}{\sqrt{\xi_{i_t}}}\mathbb{E}\left[\max_{h\in\mathcal{H}_{i_t}}\left|\bar{l}_t(h,Z) - \max_{u_{i_t}\le m\le i_t} e_m(h)\right|\right] \\
&\le \sum_{t=0}^{\infty}\sqrt{\xi_{i_t}} \\
&< \infty.
\end{aligned}$$

By the Borel-Cantelli Lemma, there exists $t_1 \in \mathbb{N}$, such that for all $t \ge t_1$,

$$\max_{h\in\mathcal{H}_{i_t}}\left|\bar{l}_t(h,Z) - \max_{u_{i_t}\le m\le i_t} e_m(h)\right| \le \sqrt{\xi_{i_t}}.$$

Let $j_t = \max\{t' : i_{t'} \le t\}$. Observe that $i_{j_t} \to \infty$ as $t \to \infty$. Since $i_{j_t} \le t$,

$$\bar{l}_t(h,Z) - \max_{u_{i_{j_t}}\le m\le t} e_m(h) \le \bar{l}_t(h,Z) - \max_{u_{i_{j_t}}\le m\le i_{j_t}} e_m(h).$$

Construct a random variable $t_2$ such that for all $t \ge t_2$,$j_t \ge t_1$. Thus, for all $t \ge t_2$,

$$\begin{aligned}
&\max_{h\in\mathcal{H}_{i_{j_t}}}\left\{\bar{l}_t(h,Z) - \max_{u_{i_{j_t}}\le m\le t} e_m(h)\right\} \\
&\le \max_{h\in\mathcal{H}_{i_{j_t}}}\left|\bar{l}_t(h,Z) - \max_{u_{i_{j_t}}\le m\le i_{j_t}} e_m(h)\right| \\
&\le \sqrt{\xi_{i_{j_t}}}.
\end{aligned}$$

Choose $i_t = i_{j_t}$ and notice that since $i_t \to \infty$, we have $i_t \le t$. With probability one, for all $t \ge t_2$,

$$\max_{h\in\mathcal{H}_{i_t}}\left\{\bar{l}_t(h,Z) - \max_{u_{i_t}\le m\le t} e_m(h)\right\} \le \sqrt{\xi_{i_t}}.$$

For $t \ge t_2$,

$$\begin{aligned}
\bar{l}_t(\hat{h}^{(t)},Z) &\le \max_{u_{i_t}\le m\le t} e_m(\hat{h}^{(t)}) + \sqrt{\xi_{i_t}} \\
&\le \max_{u_{i_t}\le m\le t} e_m(h^{(t)}) + \sqrt{\xi_{i_t}} \\
&\le \sup_{u_{i_t}\le m\le t} e_m(h^{(t)}) + \sqrt{\xi_{i_t}}
\end{aligned}$$

Hence,

$$\mathbb{E}[\bar{l}_t(\hat{h}^{(t)},Z)|Z_{\le t}] - \mathbb{E}\left[\sup_{u_{i_t}\le m\le t} e_m(h^{(t)})|Z_{\le t}\right] \to 0$$

almost surely. By Lemma 3, we have,

$$\mathbb{E}\left[\bar{l}_t(\hat{h}^{(t)}, Z)|Z_{\leq t}\right] - R_t^* \to 0.$$

Hence, by the bounded convergence theorem,

$$0 = \mathbb{E}\left[\lim_{t\to\infty}\left(\mathbb{E}\left[\bar{l}_t(\hat{h}^{(t)}, Z)|Z_{\leq t}\right] - R_t^*\right)\right] = \lim_{t\to\infty}\mathbb{E}\left[\mathbb{E}\left[\bar{l}(\hat{h}^{(t)}, Z)|Z_{\leq t}\right] - R_t^*\right].$$

## C Reinforcement Learning Baselines

In this work, we consider the Fitted Q-iteration (FQI) algorithm (Ernst et al., 2005; Munos and Szepesvari, 2008), and a time-aware variant of it as the reinforcement learning baselines. Let $s_t \in \mathcal{S}, a_t \in \mathcal{A}$ and $r_t \in \mathbb{R}$ denote the state, action, and reward at time $t$, respectively. Suppose we are given a dataset of interactions $\mathcal{D} = \{(s_t, a_t, r_t)\}_{t=1}^T$ collected using some stochastic behavior policy by interacting with a Markov Decision Process (MDP). The generic FQI algorithm approximates the state-action value function $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ by running the following iterative condition. At iteration $k$, it constructs training targets

$$y_t^{(k)} = r_t + \gamma \max_{a' \in \mathcal{A}} Q_k(s_{t+1}, a'),$$

from the dataset $\mathcal{D}$, and then it updates $Q_{k+1}$ by fitting a regressor from a model class $\mathcal{F}$ onto these targets.

$$Q_{k+1} = \operatorname*{argmin}_{f \in \mathcal{F}} \sum_{t=1}^{T} \left(f(s_t, a_t) - y_t^{(k)}\right)^2$$

Once we have the the approximated $\hat{Q}$-function, we use the greedy policy given by,

$$\pi(s) = \operatorname*{argmax}_{a \in \mathcal{A}} \hat{Q}(s, a)$$

In the time-aware variant of FQI, we make the $Q$-function to be a function of time $t$ in addition to the state $s$ and action $a$. We do this by slightly modifiying the iterative condition as follows. At iteration $k$, it constructs training targets

$$y_t^{(k)} = r_t + \gamma \max_{a' \in \mathcal{A}} Q_k(s_{t+1}, a', t),$$

from the dataset $\mathcal{D}$, and then it updates $Q_{k+1}$ by fitting a regressor from a model class $\mathcal{F}$ onto these targets.

$$Q_{k+1} = \operatorname*{argmin}_{f \in \mathcal{F}} \sum_{t=1}^{T} \left(f(s_t, a_t, t) - y_t^{(k)}\right)^2$$

As before, once we have the approximated $\hat{Q}$-function, we construct a greedy policy by,

$$\pi(s, t) = \operatorname*{argmax}_{a \in \mathcal{A}} \hat{Q}(s, a, t)$$

However, unlike the static policy from generic FQI, the time-aware variant leads to a dynamic policy that varies with time. In our experiments, we use a Random Forest with 1000 trees as the regressor of the FQI. We illustrate the peusdocode of online FQI in algorithm 2.

**Algorithm 2** Online Fitted Q-Iteration with $\epsilon$-greedy exploration

**Initialize** Foraging environment `env`, experience buffer $\mathcal{D} \leftarrow \emptyset$, warmup size $W$, update interval $I$, exploration parameter $\epsilon$, action space $\mathcal{A} = \{-1, 0, 1\}$
$\hat{Q} \leftarrow$ None
**for** $t = 1$ to $T$ **do**
    $s_t \leftarrow$ `env`.get_current_state()
    Sample $u \sim$ Uniform$[0, 1]$
    **if** $u < \max(0.01, \epsilon \cdot 0.999^t)$ or $Q$ is None **then**
        $a_t \leftarrow$ RandomChoice$(\mathcal{A})$ ▷ Explore
    **else**
        $a_t \leftarrow \text{argmax}_{a' \in \mathcal{A}} \hat{Q}(s, a', t)$ ▷ Exploit
    **end if**
    $(s_{t+1}, r_t) \leftarrow$ `env`.step$(a_t)$
    Add $(s_t, a_t, r_t, t)$ to $\mathcal{D}$
    **if** $|\mathcal{D}| \geq W$ **and** $(t + 1) \bmod I = 0$ **then**
        $\hat{Q} =$ `FQI`.fit$(\mathcal{D})$
    **end if**
**end for**

## THE USE OF LARGE LANGUAGE MODELS (LLMS)

We used a LLM model for language editing (grammar and polish) only. All scientific claims, methods, and results were produced and verified by the authors, who are fully responsible for the manuscript.